

Introducción a desarrollo web en entorno servidor

José Manuel Sánchez Álvarez

16 de septiembre de 2025

Índice de contenidos

Capítulo 1. Introducción	1
1.1. Funcionamiento de una aplicación web	1
1.2. Páginas estáticas vs dinámicas	1
1.3. Protocolo HTTP	2
1.4. Backend vs. Frontend	2
1.5. Arquitectura de Aplicaciones	2
1.6. Tecnologías Backend	3
1.7. Desarrollando con un Framework	3
1.8. Spring Framework	4
1.9. Spring Boot	4
1.10. Herramientas para Desarrolladores	4
Comprueba tu Aprendizaje	5

¡Perfecto! Vamos a ampliar el **Capítulo 1: Introducción** para incluir los temas que mencionas. A continuación, te presento la versión actualizada del capítulo con las nuevas secciones integradas.

Capítulo 1. Introducción

1.1. Funcionamiento de una aplicación web

Una aplicación web es un sistema que permite la interacción entre usuarios y servidores mediante un navegador web. Su funcionamiento básico se puede dividir en tres componentes principales:

- 1. Cliente (Frontend):**
 - Es la interfaz con la que interactúa el usuario.
 - Se construye usando HTML, CSS y JavaScript.
- 2. Servidor (Backend):**
 - Procesa las solicitudes del cliente y devuelve respuestas.
 - Ejecuta lógica de negocio, gestiona bases de datos y autenticación.
- 3. Base de Datos:**
 - Almacena la información persistente de la aplicación.

El flujo general es: - El cliente envía una solicitud HTTP al servidor. - El servidor procesa la solicitud, consulta o modifica la base de datos si es necesario. - El servidor devuelve una respuesta HTTP al cliente.

1.2. Páginas estáticas vs dinámicas

Páginas estáticas:

- Contienen contenido fijo que no cambia según las interacciones del usuario.
- Ejemplo: Un archivo HTML simple con texto e imágenes.
- Limitaciones: No permiten interactividad ni personalización.

Páginas dinámicas:

- Generan contenido de forma dinámica en función de las solicitudes del usuario.
 - Permiten interactividad y personalización.
 - Técnicas comunes:
 - **CGI (Common Gateway Interface):**
 - * Permite que un servidor ejecute scripts en lenguajes como Perl, Python o PHP para generar páginas dinámicas.
 - **Código embebido en HTML:**
 - * Lenguajes como PHP o JSP permiten incrustar código directamente en archivos HTML para generar contenido dinámico.
-

1.3. Protocolo HTTP

HTTP (HyperText Transfer Protocol) es el protocolo que define cómo se comunican los clientes y servidores en la web. Sus características clave son:

1. **Métodos HTTP:**
 - **GET:** Solicitar datos.
 - **POST:** Enviar datos al servidor.
 - **PUT/PATCH:** Actualizar datos existentes.
 - **DELETE:** Eliminar datos.
 2. **Códigos de Estado:**
 - **2xx:** Éxito.
 - **3xx:** Redirecciones.
 - **4xx:** Errores del cliente.
 - **5xx:** Errores del servidor.
 3. **Cabeceras HTTP:**
 - Proporcionan metadatos sobre la solicitud o respuesta.
-

1.4. Backend vs. Frontend

Las aplicaciones web modernas se dividen en dos partes principales:

1. **Backend:**
 - Maneja la lógica de negocio, autenticación, acceso a bases de datos y generación de respuestas.
 - Tecnologías comunes: Java (Spring), Node.js, Python (Django/Flask), PHP.
2. **Frontend:**
 - Es la parte visible para el usuario, que interactúa directamente con el navegador.
 - Tecnologías comunes: HTML, CSS, JavaScript (React, Angular, Vue.js).

El backend y el frontend se comunican a través de APIs RESTful, SOAP o GraphQL.

1.5. Arquitectura de Aplicaciones

Las aplicaciones web pueden seguir diferentes arquitecturas según su complejidad y necesidades:

1. **Monolítica:**
 - Todo el código (frontend, backend y base de datos) está en un solo proyecto.
 2. **Arquitectura Cliente-Servidor:**
 - El frontend (cliente) se comunica con el backend (servidor) mediante APIs.
 3. **Microservicios:**
 - La aplicación se divide en pequeños servicios independientes que se comunican entre sí.
 4. **Arquitectura MVC (Modelo-Vista-Controlador):**
 - **Modelo:** Gestiona los datos (base de datos).
 - **Vista:** Muestra los datos al usuario (frontend).
 - **Controlador:** Maneja la lógica de negocio y actúa como intermediario entre el modelo y la vista.
-

1.6. Tecnologías Backend

Comunicación entre cliente y servidor:

- **AJAX (Asynchronous JavaScript and XML):**
 - Permite realizar solicitudes asíncronas al servidor sin recargar la página.
 - Ejemplo: Cargar datos dinámicos en una tabla sin refrescar la página completa.
- **SOAP (Simple Object Access Protocol):**
 - Protocolo basado en XML para intercambiar datos estructurados.
 - Menos común hoy en día debido a su complejidad.
- **RESTful API:**
 - API basada en principios REST que utiliza HTTP para operaciones CRUD.
 - Ligerio, flexible y ampliamente utilizado en aplicaciones modernas.

Soluciones Backend:

1. **Desarrollo desde cero:**
 - Construir todo el backend manualmente sin frameworks.
 - Requiere más tiempo y esfuerzo, pero ofrece total control.
 2. **Frameworks:**
 - Herramientas que simplifican el desarrollo backend.
 - Ejemplos: Spring Boot (Java), Django (Python), Express (Node.js).
 3. **CMS (Content Management System):**
 - Plataformas para gestionar contenido web sin necesidad de programar.
 - Ejemplos: WordPress, Joomla, Drupal.
-

1.7. Desarrollando con un Framework

Cuando usas un framework, el desarrollo sigue una estructura clara y organizada:

1. **URLs:**
 - Define las rutas que manejan las solicitudes HTTP.
 - Ejemplo: `/api/users` para gestionar usuarios.
2. **Controladores:**
 - Contienen la lógica para procesar las solicitudes y devolver respuestas.
 - Ejemplo: Un controlador para manejar operaciones CRUD de usuarios.
3. **Modelo:**
 - Representa los datos y su interacción con la base de datos.
 - Ejemplo: Una clase `User` que mapea una tabla de usuarios en la base de datos.
4. **Plantillas:**

- Archivos que definen la estructura de las vistas dinámicas.
 - Ejemplo: Usar Thymeleaf para generar HTML dinámico en Spring Boot.
5. **Servicios:**
- Contienen la lógica de negocio y actúan como intermediarios entre controladores y modelos.
-

1.8. Spring Framework

Spring es un framework de Java ampliamente utilizado para el desarrollo de aplicaciones web y enterprise. Sus características principales incluyen:

1. **Inversión de Control (IoC):**
 - Permite gestionar las dependencias automáticamente mediante inyección de dependencias.
 2. **Programación Orientada a Aspectos (AOP):**
 - Facilita la separación de preocupaciones cross-cutting.
 3. **Soporte para múltiples capas:**
 - **Web:** Controladores RESTful.
 - **Datos:** Acceso a bases de datos mediante JPA/Hibernate.
 - **Seguridad:** Autenticación y autorización.
 4. **Modularidad:**
 - Spring se divide en módulos como Spring Web, Spring Data, Spring Security, etc.
-

1.9. Spring Boot

Spring Boot es una extensión de Spring que simplifica el desarrollo de aplicaciones:

1. **Configuración automática:**
 - Reduce la necesidad de configuraciones manuales mediante convenciones.
 2. **Empaquetado standalone:**
 - Genera archivos ejecutables (JAR) que incluyen un servidor embebido (Tomcat, Jetty).
 3. **Desarrollo rápido:**
 - Herramientas como Spring Initializr permiten crear proyectos rápidamente.
 4. **Ecosistema integrado:**
 - Soporte para bases de datos, seguridad, APIs REST y más.
-

1.10. Herramientas para Desarrolladores

Para desarrollar aplicaciones web en Java/Spring, necesitas herramientas específicas:

1. **IDE (Entorno de Desarrollo Integrado):**
 - Visual Studio Code, IntelliJ IDEA, Eclipse, NetBeans.
2. **Gestión de Dependencias:**
 - Maven o Gradle para manejar bibliotecas externas.
3. **Servidor de Base de Datos:**
 - MySQL, PostgreSQL, H2 (para desarrollo).
4. **Herramientas de Testing:**
 - JUnit, Mockito, Postman.
5. **Herramientas de Documentación:**
 - Swagger/OpenAPI para documentar APIs REST.
6. **Herramientas de Monitoreo:**
 - Actuator para monitorear aplicaciones en producción.

Comprueba tu Aprendizaje

Responde las siguientes preguntas para verificar tu comprensión:

1. ¿Qué diferencia hay entre páginas estáticas y dinámicas?
 2. Explica brevemente qué es AJAX y por qué es útil.
 3. ¿Qué ventajas ofrecen los frameworks backend como Spring Boot?
 4. Nombra tres características clave del protocolo HTTP.
 5. ¿Qué papel juega el modelo en la arquitectura MVC?
-